Lecture 18: Post-Training

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 23, 2026

WILLIAM & MARY

Many slides are adapted from Jurafsky & Martin (3rd ed.) & Chris Manning& Swabha Swayamdipta

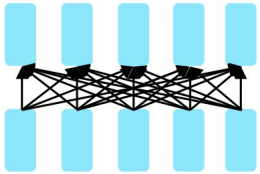
Lecture Outline

- Pretraining: Architectures, & Objectives
- Post-Training:
  - Instruction Tuning
  - Preference Tuning (RLHF - not covered, DPO)

Architectures (and Objectives) of Pretrained Transformer Models

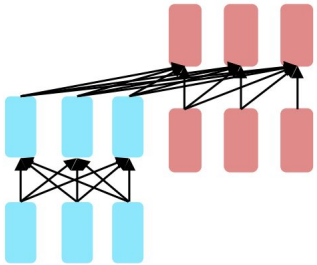
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



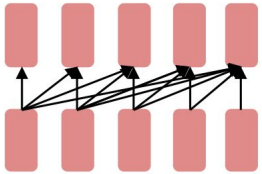
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Combines the encoder's bidirectional understanding of the input with the decoder's ability to generate output autoregressively.
- What's the best way to pretrain them?



Decoders

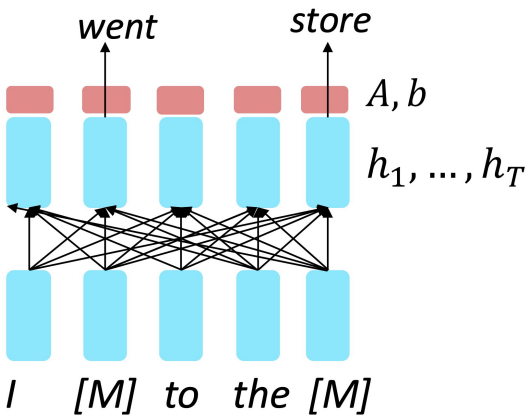
- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining encoders: what pretraining objective to use?

So far, we've looked at language model pretraining. But **encoders get bidirectional context**, so we can't do language modeling!

Idea: replace some fraction of words in the input with a special [MASK] token; predict these words.

$$h_1, \dots, h_T = \text{Encoder}(w_1, \dots, w_T)$$
$$y_i \sim Aw_i + b$$



Only add loss terms from words that are "masked out." If $\tilde{x}$ is the masked version of x, we're learning $p_\theta(x|\tilde{x})$. Called **Masked LM**.

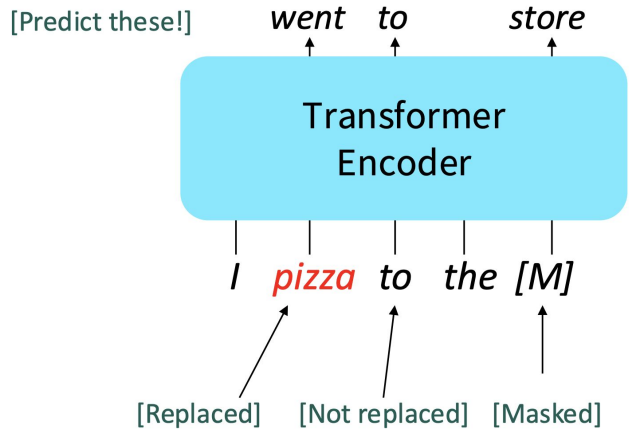
[Devlin et al., 2018]

BERT: Bidirectional Encoder Representations from Transformers

Devlin et al., 2018 proposed the "Masked LM" objective and **released the weights of a pretrained Transformer**, a model they labeled BERT.

Some more details about Masked LM for BERT:

- Predict a random 15% of (sub)word tokens.
  - Replace input word with [MASK] 80% of the time
  - Replace input word with a random token 10% of the time
  - Leave input word unchanged 10% of the time (but still predict it!)

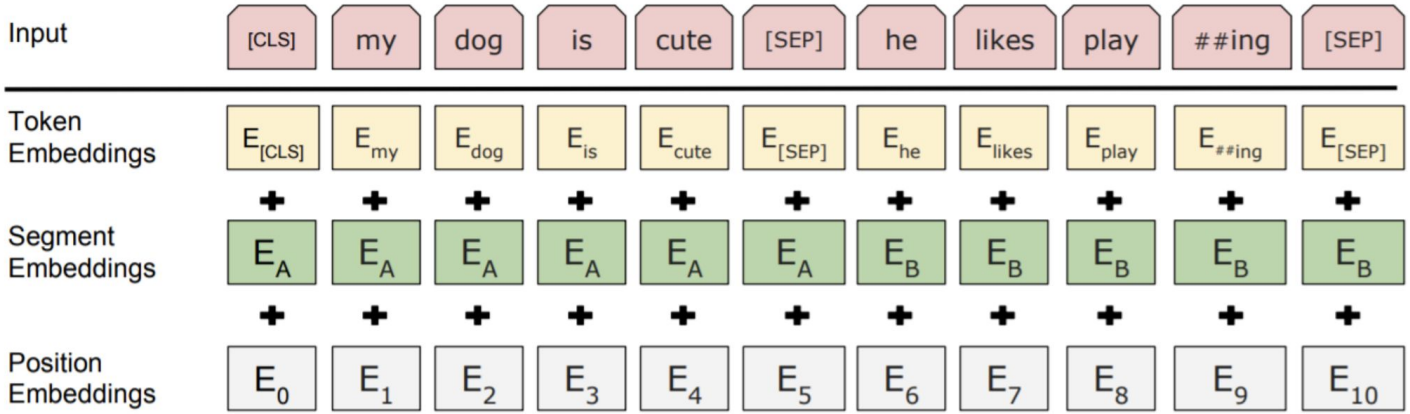


[Devlin et al., 2018]

Why? Doesn't let the model get complacent and not build strong representations of non-masked words. (No masks are seen at fine-tuning time!)

BERT: Bidirectional Encoder Representations from Transformers

- The pretraining input to BERT was two separate contiguous chunks of text:



- BERT was trained to predict whether one chunk follows the other or is randomly sampled.
  - • [CLS] and [SEP] tokens
  - • [SEP] is used for next sentence prediction - do these sentences follow each other?
  - • [CLS] for text classification

BERT: Bidirectional Encoder Representations from Transformers

Details about BERT

- Two models were released:
  - BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- Trained on:
  - BooksCorpus (800 million words)
  - English Wikipedia (2,500 million words)
- Pretraining is expensive and impractical on a single GPU.
  - BERT was pretrained with 64 TPU chips for a total of 4 days.
  - (TPUs are special tensor operation acceleration hardware)
- Finetuning is practical and common on a single GPU
  - “Pretrain once, finetune many times.”



BERT: Bidirectional Encoder Representations from Transformers

BERT was massively popular and hugely versatile; finetuning BERT led to new state-of the-art results on a broad range of tasks.

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

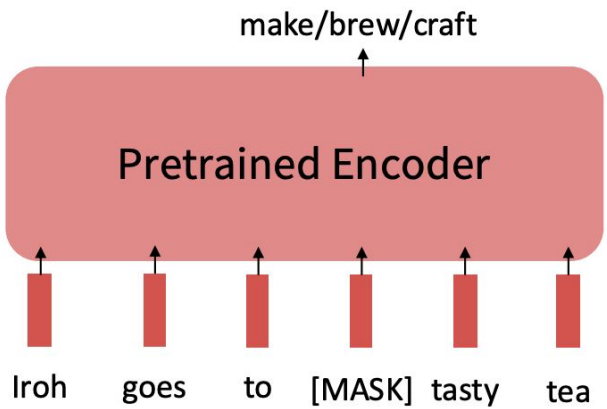
- **QQP:** Quora Question Pairs (detect paraphrase questions)
- **QNLI:** natural language inference over question answering data
- **SST-2:** sentiment analysis

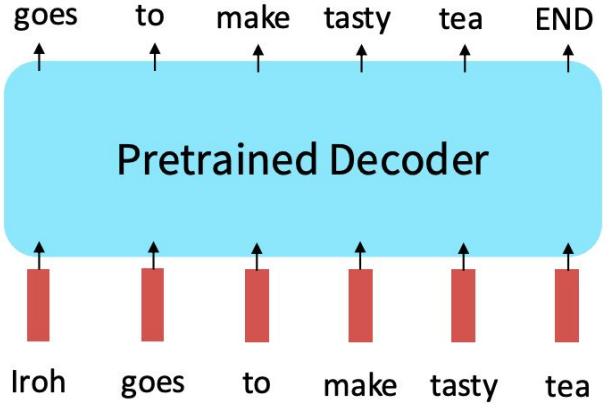
- **CoLA:** corpus of linguistic acceptability (detect whether sentences are grammatical.)
- **STS-B:** semantic textual similarity
- **MRPC:** microsoft paraphrase corpus
- **RTE:** a small natural language inference corpus

Limitations of Pretrained Encoders

Those results looked great! Why not used pretrained encoders for everything?

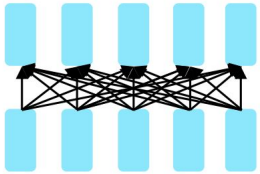
If your task involves generating sequences, consider using a pretrained decoder; BERT and other pretrained encoders don't naturally lead to nice autoregressive (1-word-at-a-time) generation methods.





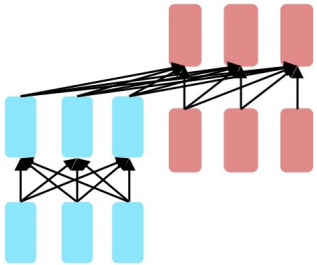
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



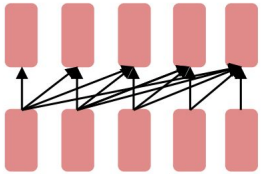
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Combines the encoder's bidirectional understanding of the input with the decoder's ability to generate output autoregressively.
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Pretraining decoders: what pretraining objective to use?

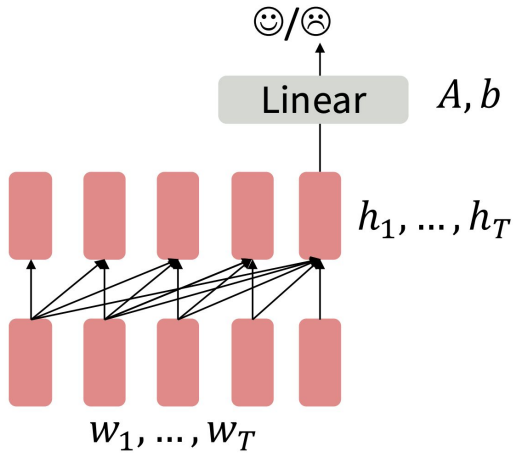
When using language model pretrained decoders, we can ignore that they were trained to model $p(w_t|w_{1:t-1})$.

We can finetune them by training a classifier on the last word's hidden state.

$$h_1, \dots, h_T = \text{Decoder}(w_1, \dots, w_T)$$
$$y \sim Ah_T + b$$

Where A and b are randomly initialized and specified by the downstream task.

Gradients backpropagate through the whole network.



[Note how the linear layer hasn't been pretrained and must be learned from scratch.]

Pretraining decoders: what pretraining objective to use?

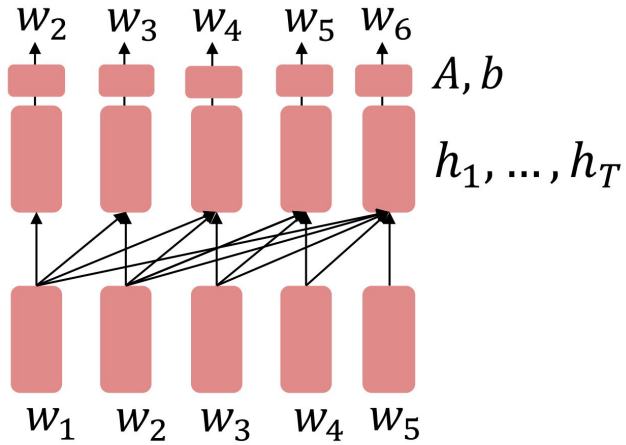
It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p_\theta(w_t|w_{1:t-1})$.

This is helpful in tasks where the output is a sequence with a vocabulary like that at pretraining time!

- Dialogue (context=dialogue history)
- Summarization (context=document)

$$h_1, \ldots, h_T = \text{Decoder}(w_1, \ldots, w_T)$$
$$w_t \sim Ah_{t-1} + b$$

Where A and b are randomly initialized and specified by the downstream task.



[Note how the linear layer has been pretrained.]

Generative Pretrained Transformer (GPT)

2018's GPT was a big success in pretraining a decoder!

- Transformer decoder with 12 layers, 117M parameters.
- 768-dimensional hidden states, 3072-dimensional feed-forward hidden layers.
- Byte-pair encoding with 40,000 merges
- Trained on BooksCorpus: over 7000 unique books.
- Contains long spans of contiguous text, for learning long-distance dependencies.

[Radford et al., 2018]

Generative Pretrained Transformer (GPT)

How do we format inputs to our decoder for finetuning tasks?

Natural Language Inference (NLI): Label pairs of sentences as *entailing/contradictory/neutral*

Premise: *The man is in the doorway*
Hypothesis: *The person is near the door*
} **entailment**

Radford et al., 2018 evaluate on natural language inference.
Here's roughly how the input was formatted, as a sequence of tokens for the decoder:

[START] *The man is in the doorway* [DELIM] *The person is near the door* [EXTRACT]

The linear classifier is applied to the representation of the [EXTRACT] token.

Generative Pretrained Transformer (GPT)

GPT results on various *natural language inference* datasets.

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	89.3	-	-	-
CAFE [58] (5x)	80.2	79.0	89.3	-	-	-
Stochastic Answer Network [35] (3x)	80.6	80.1	-	-	-	-
CAFE [58]	78.7	77.9	88.5	83.3		
GenSen [64]	71.4	71.3	-	-	82.3	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	**61.7**
Finetuned Transformer LM (ours)	**82.1**	**81.4**	**89.9**	**88.3**	**88.1**	56.0

GPT-2

- GPT-2, a larger version (1.5B) of GPT trained on more data, was shown to produce relatively convincing samples of natural language.
- Moved away from classification, only generation

Context (human-written): In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.
GPT-2: The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science. Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved. Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.

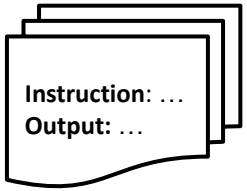
Post-Training

LLM Training Stages



Self-supervised learning on large-scale, (often proprietary) unlabeled data (often web text).

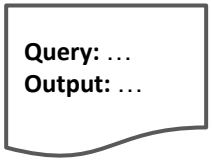
Pretraining



Supervised (e.g. RLHF).

- Task-specific data
- Alignment with users
- Instruction tuning
- Few-shot learning

Post-Training & Fine-tuning
(weight update)



Designing queries for model to elicit desired output.

- In-context learning
- Prompt engineering

Prompting
(no weight update)

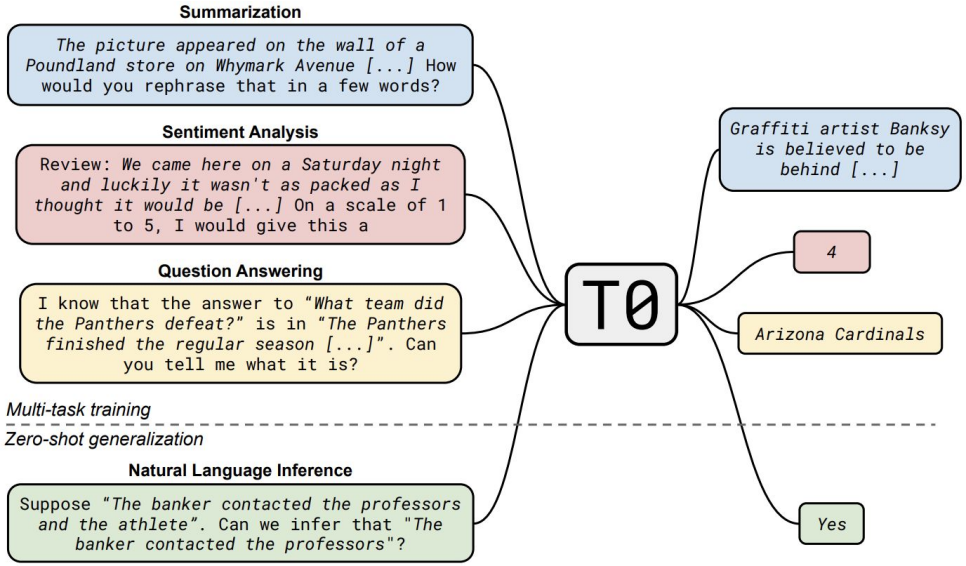
Pretraining and Post-Training

- **Pre-training:** Self-supervised, standard next token prediction
  - Needs lots of raw text:
    - Crawled from the Web: Natural Solution
    - But not all data crawled from the web is good for LM training
    - Needs to be filtered in various ways (deduplicated, removing non-natural language like code, sentences with offensive words from a blocklist).
- **Post-training:** Supervision (sequence-to-sequence)
  - I**nstruction-Tuning:** Supervision is not necessarily via labels, but sequence pairs.
    - Labels in standard NLP benchmarks can be converted into sequence pairs (repurposed NLP/ML benchmarks)
  - **Preference-Tuning:** Collects human judgments / preferences as rewards, or a pair of preferred / dispreferred generations
    - Often human labels, not easy to produce

Post-Training: Instruction-Tuning

Instruction-Tuning

- Pretraining:
  - Train a model to continue a given context
- Instruction Tuning:
  - Train a model to follow varied instructions
  - Needed because the vast majority of pretraining is done on data which are not in the form of instructions
  - Fine-tuned (using the next-token prediction objective) on a dataset of instructions together with correct responses



(Sanh et al., 2022)

Instruction Tuning Data: From Templates

Extracting Instructions from NLP Datasets

Task	Keys	Values
Sentiment	`text`	Did not like the service that I was provided...
	`label`	0
NLI	`premise`	Jimmy Smith... played college football at University of Colorado.
	`hypothesis`	The University of Colorado has a college football team.
	`label`	0
Extractive Q/A	`context`	Beyoncé Giselle Knowles-Carter is an American singer...
	`question`	When did Beyoncé start becoming popular?
	`answers`	{ `text`: ['in the late 1990s'], `answer_start`: 269 }

Figure 8.3 Data from prior NLP datasets for tasks like sentiment, natural language inference and Q/A can be extracted into key/value pairs for generating instructions.

Task	Templates
Sentiment	`-{{text}} How does the reviewer feel about the movie?`
	`-{{text}} Did the reviewer enjoy the movie?`
Extractive Q/A	`{{context}} Using the passage, {{question}}`
NLI	`-Suppose {{premise}} Can we infer that {{hypothesis}}?`
	`Yes, no, or maybe?`

Figure 8.4 Instruction templates for sentiment, Q/A and NLI tasks.

Instruction-Tuning and Task Generalization

- During instruction tuning, the model learns to follow instructions of given tasks
- At test time, it generalizes to follow instructions on unseen tasks!
- Labels from humans or LLMs

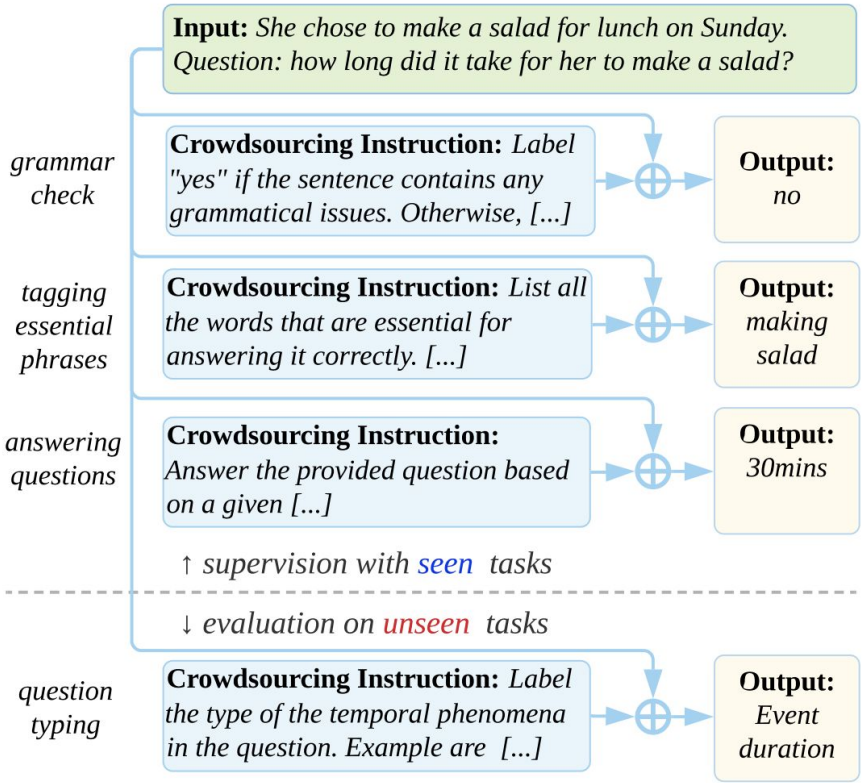


Figure 1: We construct the NATURAL INSTRUCTIONS dataset from crowdsourcing instructions and instances of different NLP datasets. We study if models can learn from *seen* tasks and generalize to *unseen* tasks given their natural crowdsourcing instructions.

Annotation Instructions

Sample Extended Instruction

- **Definition**: In this task we ask you to write answer to a question that involves "absolute timepoint" of events, which is defined as understanding of when events usually happen. For example, "going to school" usually happens during the day (not at 2 A.M). *
- **Emphasis**: Note that a lot of the questions could have more than one correct answers. We only need a single most-likely answer. Please try to keep your "answer" as simple as possible. Concise and simple "answer" is preferred over those complex and verbose ones. **Prompt**: Answer the given question on "absolute timepoint" of events.
  Sentence: { sentence }
  Question: { question }

Figure 8.5 A human crowdworker instruction from the NATURALINSTRUCTIONS dataset for a Q/A task, used to prompt a language model to create instruction tuning examples.

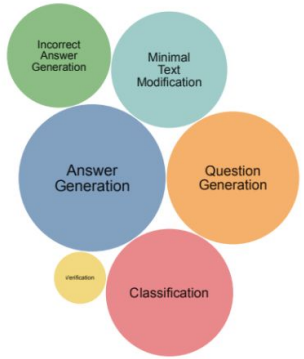
(Wang et al., 2022)

Diversity of Instruction Data

How to generalize to variety of unseen tasks? Train on more diverse data.



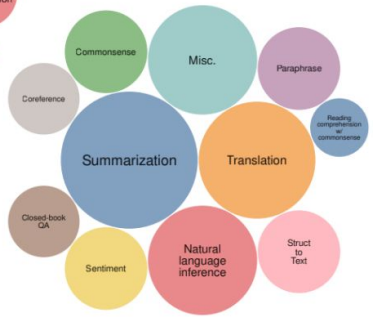
(a) SUP-NATINST (this work)



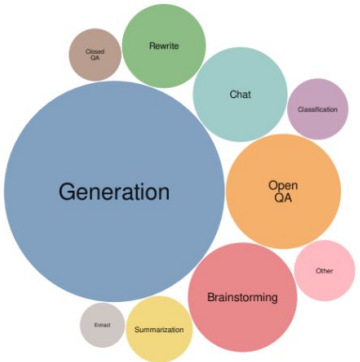
(b) NATINST



(c) PROMPTSOURCE (T0 subset)



(d) FLAN



(e) INSTRUCTGPT

Instruction-Tuning Data

- Instruction tuning datasets are often created by repurposing standard NLP datasets for tasks like question answering or machine translation
- Often synthesized!
  - Prompting existing LLMs
- More variety in the instruction templates → better models
- More data, more diverse data = better

		Model Details				Data Collection & Training Details			
Release	Collection	Model	Base	Size	Public?	Prompt Types	Tasks in Flan	# Exs	Methods
2020 05	UnifiedQA	UnifiedQA	RoBerta	110-340M	P	ZS	46 / 46	750k	
2021 04	CrossFit	BART-CrossFit	BART	140M	NP	FS	115 / 159	71.M	
2021 04	Natural Inst v1.0	Gen. BART	BART	140M	NP	ZS / FS	61 / 61	620k	+ Detailed k-shot Prompts
2021 09	Flan 2021	Flan-LaMDA	LaMDA	137B	NP	ZS / FS	62 / 62	4.4M	+ Template Variety
2021 10	P3	T0, T0+, T0++	T5-LM	3-11B	P	ZS	62 / 62	12M	+ Template Variety + Input Inversion
2021 10	MetaICL	MetaICL	GPT-2	770M	P	FS	100 / 142	3.5M	+ Input Inversion + Noisy Channel Opt
2021 11	ExMix	ExT5	T5	220M-11B	NP	ZS	72 / 107	500k	+ With Pretraining
2022 04	Super-Natural Inst.	*Tk*-Instruct	T5-LM, mT5	11-13B	P	ZS / FS	1556 / 1613	5M	+ Detailed k-shot Prompts + Multilingual
2022 10	GLM	GLM-130B	GLM	130B	P	FS	65 / 77	12M	+ With Pretraining + Bilingual (en, zh-cn)
2022 11	xP3	BLOOMz, mT0	BLOOM, mT5	13-176B	P	ZS	53 / 71	81M	+ Massively Multilingual
2022 12	Unnatural Inst.†	T5-LM-Unnat. Inst.	T5-LM	11B	NP	ZS	~20 / 117	64k	+ Synthetic Data
2022 12	Self-Instruct†	GPT-3 Self Inst.	GPT-3	175B	NP	ZS	Unknown	82k	+ Synthetic Data + Knowledge Distillation
2022 12	OPT-IML Bench†	OPT-IML	OPT	30-175B	P	ZS + FS, CoT	~2067 / 2207	18M	+ Template Variety + Input Inversion + Multilingual
2022 10	Flan 2022 (ours)	Flan-T5, Flan-PaLM	T5-LM, PaLM	10M-540B	P NP	ZS + FS, CoT	1836	15M	+ Template Variety + Input Inversion + Multilingual

(Longpre et al., 2023)

Self-Generated Instructions

(Wang et al., 2023)

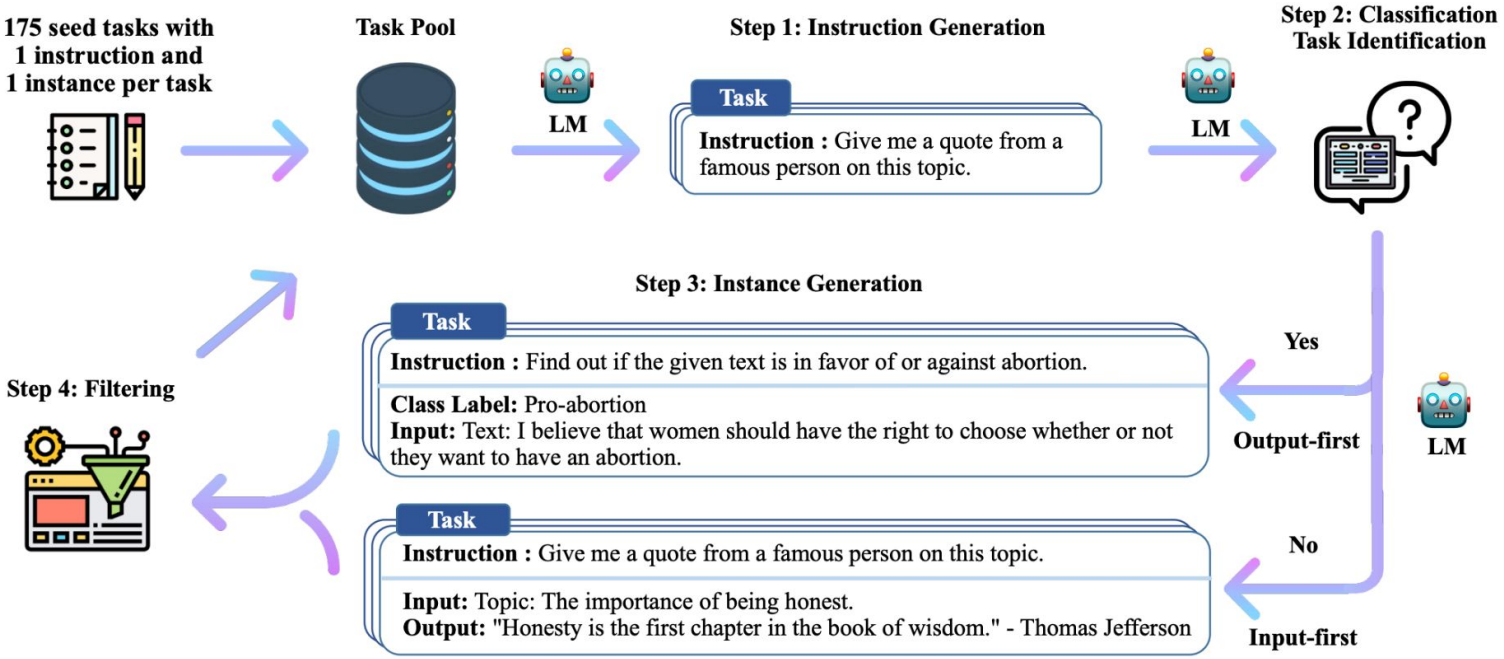


Figure 2: A high-level overview of SELF-INSTRUCT. The process starts with a small seed set of tasks as the task pool. Random tasks are sampled from the task pool, and used to prompt an off-the-shelf LM to generate both new instructions and corresponding instances, followed by filtering low-quality or similar generations, and then added back to the initial repository of tasks. The resulting data can be used for the instruction tuning of the language model itself later to follow instructions better. Tasks shown in the figure are generated by GPT3.

Instruction-Tuning: Masking Instructions

- We're still using decoder-only models (the same as we used in pre-training)
- How to update these models for an encoder-decoder like behavior?
- The instruction itself is masked, so the model does not generate instructions

In the loss function, mask out the tokens corresonding to prompt

```
Below is an instruction that describes a task. Write a
response that appropriately completes the request.

### Instruction:
Rewrite the following sentence using passive voice.

### Input:
The team achieved great results.

### Response:
Great results were achieved by the team.
```

Calculate the loss only over the tokens corresponding to the response text

An illustration of input masking where the highlighted text is still fed to the LLM, but it is not used when calculating the loss during training.

